| | |
|---|---|
| **Status** | Finished |
| **Started** | Monday, 19 October 2026, 8:42 PM |
| **Completed** | Monday, 19 October 2026, 8:54 PM |
| **Duration** | 11 mins 33 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

Question **1**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

**For example:**

| Input | Result |
|---|---|
| input1.txt | Longest word: containing |

**Answer:** (penalty regime: 0 %)

```
import sys
import re

def find_longest_word():
    try:
        file_path = input().strip()
        if not file_path:
            return

        with open(file_path, 'r', encoding='utf-8') as file:
            text = file.read()
            words = re.findall(r'\b\w+\b', text)
            if words:
                longest_word = max(words, key=len)
                print(f"Longest word: {longest_word}")
    except (EOFError, FileNotFoundError):
        pass

if __name__ == "__main__":
    find_longest_word()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | input1.txt | Longest word: containing | Longest word: containing | ✓ |
| ✓ | input2.txt | Longest word: thousand | Longest word: thousand | ✓ |
| ✓ | input3.txt | Longest word: supercalifragilisticexpialidocious | Longest word: supercalifragilisticexpialidocious | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

**For example:**

| Input | Result |
|---|---|
| input1.txt<br>3 | Line three. |

**Answer:** (penalty regime: 0 %)

```
import sys

def read_specific_line():
    file_path = input().strip()
    target_line_num = int(input().strip())

    try:
        with open(file_path, 'r', encoding='utf-8') as file:

            for current_line_num, line in enumerate(file, start=1):
                if current_line_num == target_line_num:
                    print(line.rstrip('\n'))
                    break
    except (FileNotFoundError, ValueError):
        pass

if __name__ == "__main__":
    read_specific_line()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | input1.txt<br>3 | Line three. | Line three. | ✓ |
| ✓ | input2.txt<br>3 | Line C. | Line C. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

**Example:**

- **Input File Content:**

  **"Line one.**
  **Line two.**
  **Line three.**
  **Line four."**
  **2**

  **Updated line two.**

**Output:**

Line one.
Updated line two.
Line three.
Line four.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`2`<br>`Updated line two.` | `Line one.`<br>`Updated line two.`<br>`Line three.`<br>`Line four.` |

**Answer:** (penalty regime: 0 %)

```
import sys

def update_line_in_file():
    file_path = sys.stdin.readline().strip()
    line_number = int(sys.stdin.readline().strip())
    new_content = sys.stdin.readline().rstrip('\r\n')

    with open(file_path, 'r', encoding='utf-8') as file:
        lines = file.readlines()

    if 1 <= line_number <= len(lines):
        lines[line_number - 1] = new_content + '\n'

    with open('output.txt', 'w', encoding='utf-8') as file:
        file.writelines(lines)

if __name__ == "__main__":
    update_line_in_file()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`2`<br>`Updated line two.` | `Line one.`<br>`Updated line two.`<br>`Line three.`<br>`Line four.` | `Line one.`<br>`Updated line two.`<br>`Line three.`<br>`Line four.` | ✓ |
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input2.txt`<br>`2`<br>`Line B Updated.` | `Line A.`<br>`Line B Updated.`<br>`Line C.` | `Line A.`<br>`Line B Updated.`<br>`Line C.` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

**Description:**

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

**For example:**

| Input | Result |
|---|---|
| input2.txt | Total words: 14 |
| input3.txt | Total words: 15 |

**Answer:** (penalty regime: 0 %)

```
import sys
import re

def count_words():
    file_path = sys.stdin.read().strip()

    if not file_path:
        return

    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            text = file.read()
            words = re.findall(r'\b\w+\b', text)
            print(f"Total words: {len(words)}")
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    count_words()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | input1.txt | Total words: 6 | Total words: 6 | ✓ |
| ✓ | input2.txt | Total words: 14 | Total words: 14 | ✓ |
| ✓ | input3.txt | Total words: 15 | Total words: 15 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

**Description:**

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output1.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`output1.txt` | `This is the source file.`<br>`It contains multiple lines of text.`<br>`Here is another line.` |

**Answer:** (penalty regime: 0 %)

```
import sys

def copy_file_contents():
    source_file = sys.stdin.readline().strip()
    destination_file = sys.stdin.readline().strip()

    if not source_file or not destination_file:
        return

    try:
        with open(source_file, 'r', encoding='utf-8') as src:
            content = src.read()

        with open(destination_file, 'w', encoding='utf-8') as dest:
            dest.write(content)
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    copy_file_contents()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output1.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`output1.txt` | `This is the source file.`<br>`It contains multiple lines of text.`<br>`Here is another line.` | `This is the source file.`<br>`It contains multiple lines of text.`<br>`Here is another line.` | ✓ |
| ✓ | `with open('output2.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input2.txt`<br>`output2.txt` | `Hello, world!`<br>`Python programming is amazing.`<br>`Let's copy this text to another file.` | `Hello, world!`<br>`Python programming is amazing.`<br>`Let's copy this text to another file.` | ✓ |
| ✓ | `with open('output3.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input3.txt`<br>`output3.txt` | `Single line.` | `Single line.` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

**Description:**

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt | madam<br>arora<br>malayalam |

**Answer:** (penalty regime: 0 %)

```
import sys
import re

def find_palindromes():
    file_path = sys.stdin.read().strip()
    if not file_path:
        return

    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            text = file.read()
            words = re.findall(r'\b\w+\b', text)

            palindromes = []
            for word in words:
                if len(word) > 1 and word.lower() == word.lower()[::-1]:
                    palindromes.append(word)

        with open('output.txt', 'w', encoding='utf-8') as output_file:
            for word in palindromes:
                output_file.write(word + '\n')
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    find_palindromes()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt | madam<br>arora<br>malayalam | madam<br>arora<br>malayalam | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

**Example:**

- **Input File Content:**

  **"Line one.**
  **Line two.**
  **Line three.**
  **Line four."**
  **3**

**Output:**

Line one.
Line two.
eerht eniL.
Line four.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`3` | `Line one.`<br>`Line two.`<br>`eerht eniL.`<br>`Line four.` |

**Answer:** (penalty regime: 0 %)

```
import sys
import re

def reverse_line_in_file():
    file_path = sys.stdin.readline().strip()
    line_number = int(sys.stdin.readline().strip())

    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            lines = file.readlines()

        if 1 <= line_number <= len(lines):
            line = lines[line_number - 1].rstrip('\r\n')

            # Separate the main text from trailing punctuation
            match = re.match(r'^(.*?)([\.\!\?\,]*)$', line)
            if match:
                text_part, punct_part = match.groups()
                lines[line_number - 1] = text_part[::-1] + punct_part + '\n'

        with open('output.txt', 'w', encoding='utf-8') as file:
            file.writelines(lines)
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    reverse_line_in_file()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt<br>3 | Line one.<br>Line two.<br>eerht eniL.<br>Line four. | Line one.<br>Line two.<br>eerht eniL.<br>Line four. | ✓ |
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input2.txt<br>2 | Line A.<br>B eniL.<br>Line C. | Line A.<br>B eniL.<br>Line C. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

**Example:**

- **Input File Content:**

  **"Line one.**
  **Line two.**
  **Line three.**
  **Line four."**
  **2**

  **Updated line two.**

**Output:**

Line one.
Line three.
Line four.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt<br>2 | Line one.<br>Line three.<br>Line four. |

**Answer:** (penalty regime: 0 %)

```
import sys

def delete_line_from_file():
    file_path = sys.stdin.readline().strip()
    line_number = int(sys.stdin.readline().strip())

    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            lines = file.readlines()

        if 1 <= line_number <= len(lines):
            del lines[line_number - 1]

        with open('output.txt', 'w', encoding='utf-8') as file:
            file.writelines(lines)
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    delete_line_from_file()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input1.txt<br>2 | Line one.<br>Line three.<br>Line four. | Line one.<br>Line three.<br>Line four. | ✓ |

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | input2.txt<br>3 | Line A.<br>Line B. | Line A.<br>Line B. | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

**Description:**

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

**Example:**

- **Input File Content:**

  apple orange apple banana apple orange

**Output:**

apple: 3
orange: 2
banana: 1

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `apple orange apple banana apple orange` | `apple: 3`<br>`banana: 1`<br>`orange: 2` |

**Answer:** (penalty regime: 0 %)

```
import sys
import re
from collections import Counter

def count_word_frequency():
    input_text = sys.stdin.read().strip()
    if not input_text:
        return

    words = re.findall(r'\b\w+\b', input_text.lower())
    counts = Counter(words)

    sorted_words = sorted(counts.keys())

    with open('output.txt', 'w', encoding='utf-8') as file:
        for word in sorted_words:
            file.write(f"{word}: {counts[word]}\n")

if __name__ == "__main__":
    count_word_frequency()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r')`<br>`as file:`<br>`  text = file.read()`<br>`  print(text)` | `apple orange apple banana apple orange` | `apple: 3`<br>`banana: 1`<br>`orange: 2` | `apple: 3`<br>`banana: 1`<br>`orange: 2` | ✓ |

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r')`<br>`as file:`<br>`  text = file.read()`<br>`  print(text)` | `Hello world! Hello everyone. Welcome to the world of programming.` | `everyone: 1`<br>`hello: 2`<br>`of: 1`<br>`programming: 1`<br>`the: 1`<br>`to: 1`<br>`welcome: 1`<br>`world: 2` | `everyone: 1`<br>`hello: 2`<br>`of: 1`<br>`programming: 1`<br>`the: 1`<br>`to: 1`<br>`welcome: 1`<br>`world: 2` | ✓ |
| ✓ | `with open('output.txt', 'r')`<br>`as file:`<br>`  text = file.read()`<br>`  print(text)` | `One fish two fish Red fish blue fish` | `blue: 1`<br>`fish: 4`<br>`one: 1`<br>`red: 1`<br>`two: 1` | `blue: 1`<br>`fish: 4`<br>`one: 1`<br>`red: 1`<br>`two: 1` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

**Description:**

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

**Example:**

- **Input File Content:**

  **"Line one.**
  **Line two.**
  **Line three.**
  **Line four."**
  **3**

  **Inserted line..**

**Output:**

Line one.
Line two.
Inserted line.
Line three.
Line four.

**For example:**

| Test | Input | Result |
|---|---|---|
| `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`3`<br>`Inserted line.` | `Line one.`<br>`Line two.`<br>`Inserted line.`<br>`Line three.`<br>`Line four.` |

**Answer:** (penalty regime: 0 %)

```
import sys

def insert_line_in_file():
    file_path = sys.stdin.readline().strip()
    line_number = int(sys.stdin.readline().strip())
    new_content = sys.stdin.readline().rstrip('\r\n')

    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            lines = file.readlines()

        lines = [line if line.endswith('\n') else line + '\n' for line in lines]

        insert_index = line_number - 1
        if insert_index >= len(lines):
            lines.append(new_content + '\n')
        else:
            lines.insert(max(0, insert_index), new_content + '\n')

        with open('output.txt', 'w', encoding='utf-8') as file:
            file.writelines(lines)
    except FileNotFoundError:
        pass

if __name__ == "__main__":
    insert_line_in_file()
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input1.txt`<br>`3`<br>`Inserted line.` | `Line one.`<br>`Line two.`<br>`Inserted line.`<br>`Line three.`<br>`Line four.` | `Line one.`<br>`Line two.`<br>`Inserted line.`<br>`Line three.`<br>`Line four.` | ✓ |
| ✓ | `with open('output.txt', 'r') as file:`<br>`  text = file.read()`<br>`  print(text)` | `input2.txt`<br>`4`<br>`Inserted line D.` | `Line A.`<br>`Line B.`<br>`Line C.`<br>`Inserted line D.` | `Line A.`<br>`Line B.`<br>`Line C.`<br>`Inserted line D.` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.